UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 2: Introduction to CUDA C and Data Parallel Programming

Volodymyr Kindratenko

1/22/2026

Course Reminders

- **Delta Access and Lab 0 Deadline**
  - **Due:** Friday at 8:00 PM (US Central Time)
  - **Lab 0** is designed to help you:
    - Get familiar with the programming environment
    - Understand the submission process
  - It's an easy lab, but setting up the tools may take some time.
  - **Important:**
    - Missing the deadline is acceptable for Lab 0, but you **must submit it eventually**.
    - Lab 0 **does not count toward your overall grade**.
- **About Delta access**
  - Yesterday, I added **~200 accounts** to the Delta class project on ACCESS.
  - However, ~**100 students** have not yet provided their ACCESS IDs to me, so their Delta accounts have not been created.
  - If you were added to the Delta group, you should see the project listed on the ACCESS website: **CIS240004: Applied Parallel Programming Course**

Today's Objectives

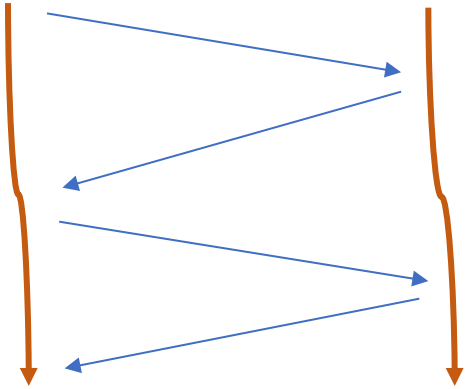
- To learn the basic concept of data parallel computing
- To learn the basic features of the CUDA C programming interface

Thread as a basic unit of computing

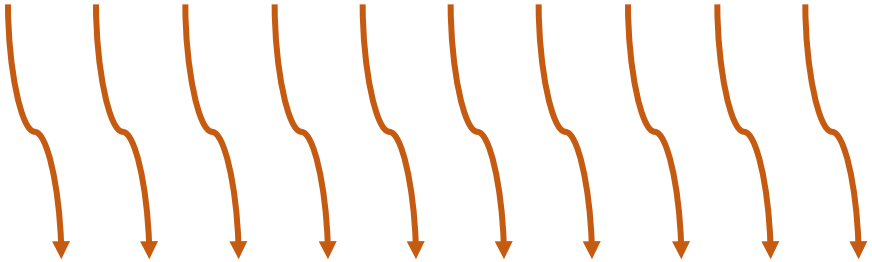
What is a thread?

- Program
- PC
- Context
  - Memory
  - Registers
- …

Multiple threads



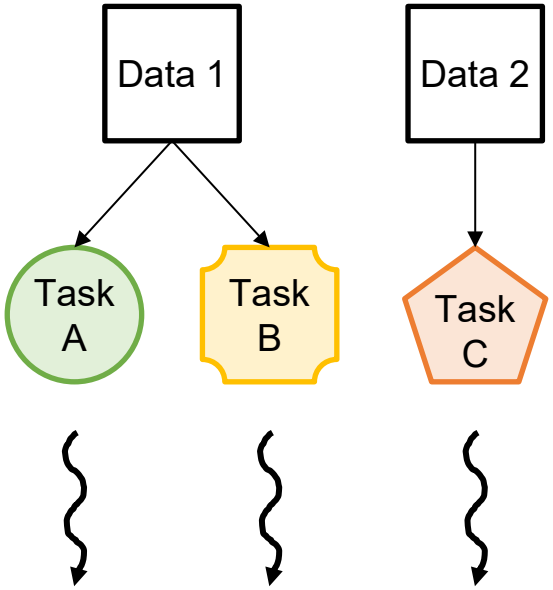
Many threads



Types of Parallelism

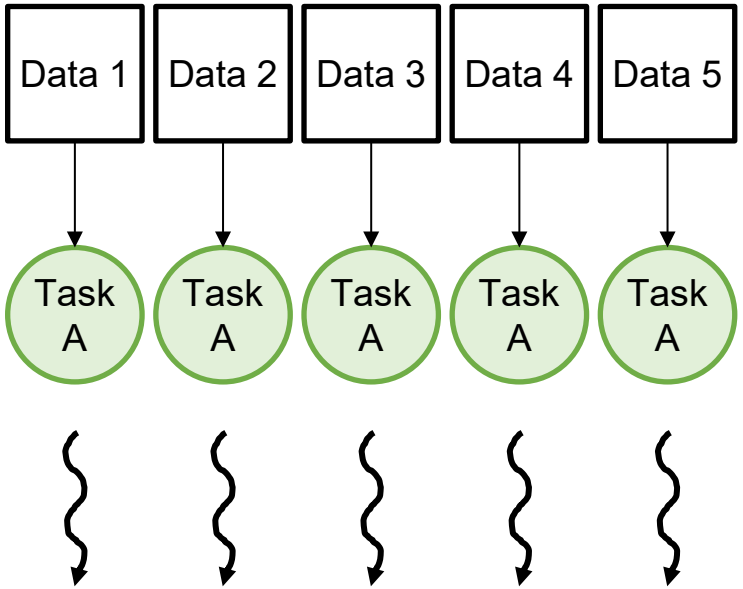
Task Parallelism

- Different operations performed on same or different data
- Usually, a modest number of tasks unleashing a modest amount of parallelism



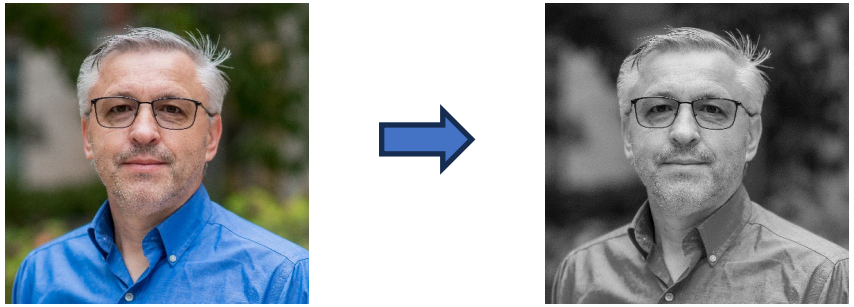
Data Parallelism

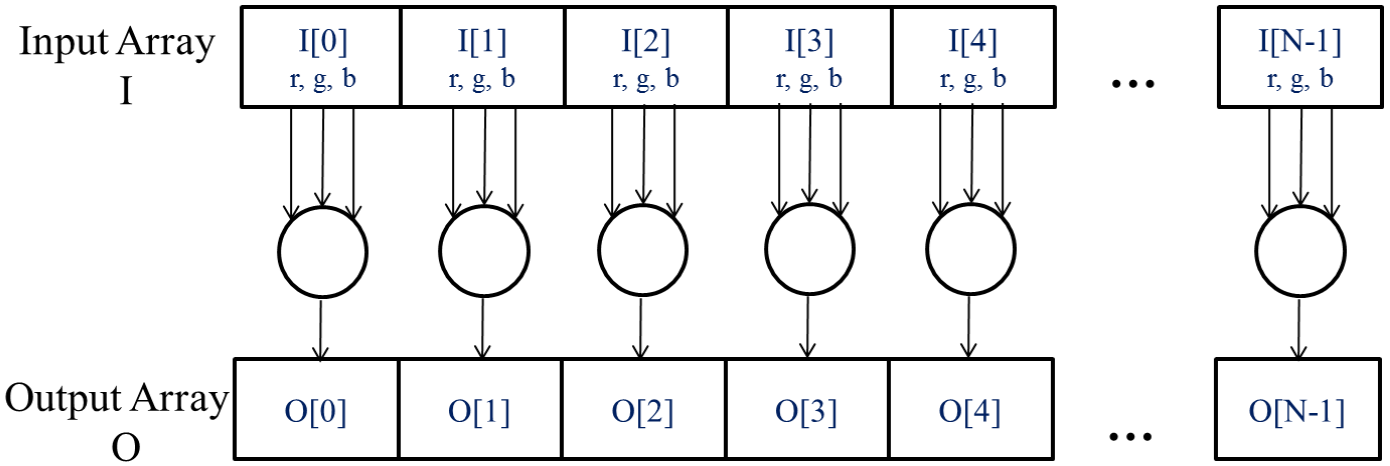
- Same operations performed on different data
- Potentially massive amounts of data unleashing massive amounts of parallelism
  - Most suitable for GPUs



A Data Parallel Computation Example

- Conversion of a color image to grey-scale image

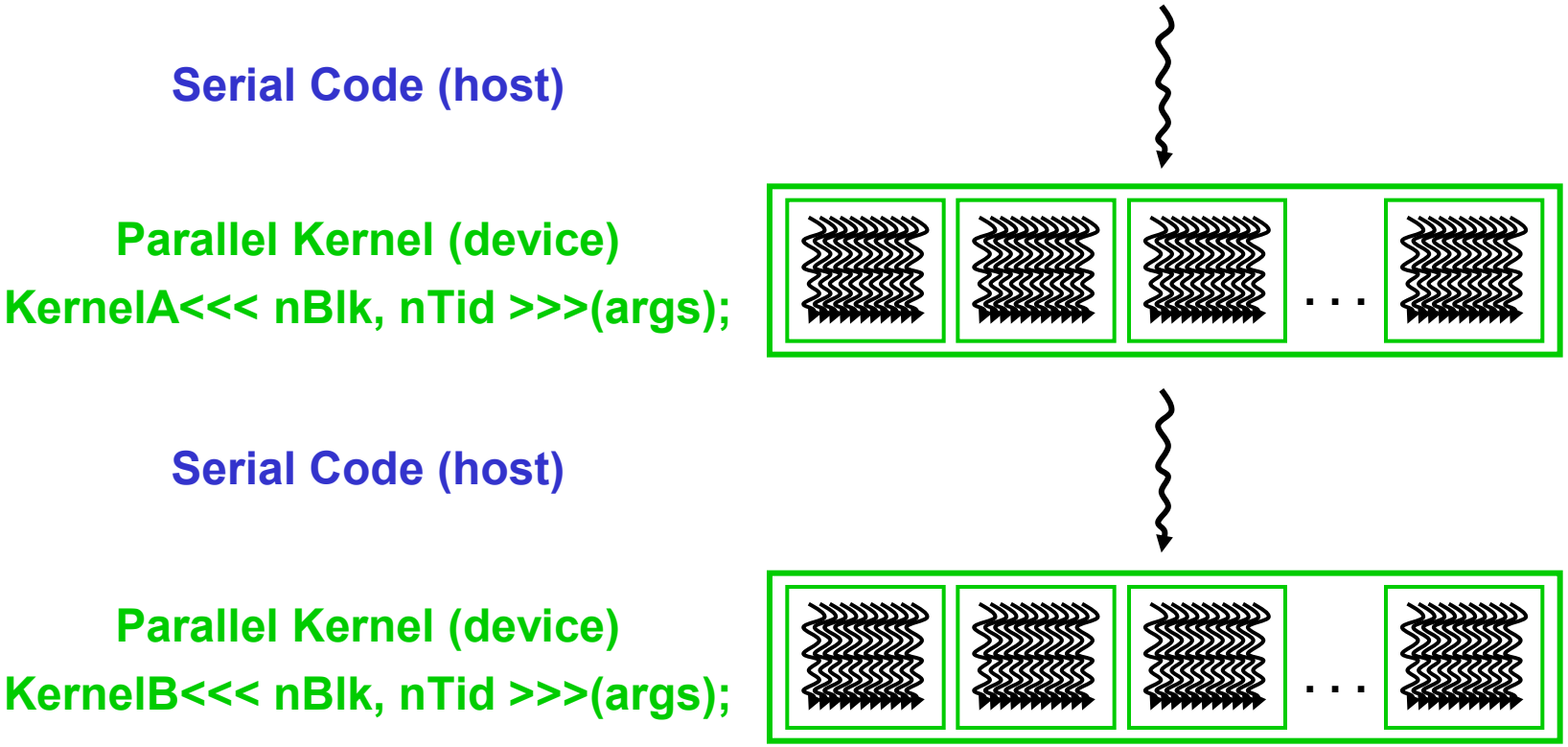





```
for each pixel {
     pixel = gsConvert(pixel)
}
// Every pixel is independent of every other pixel
```

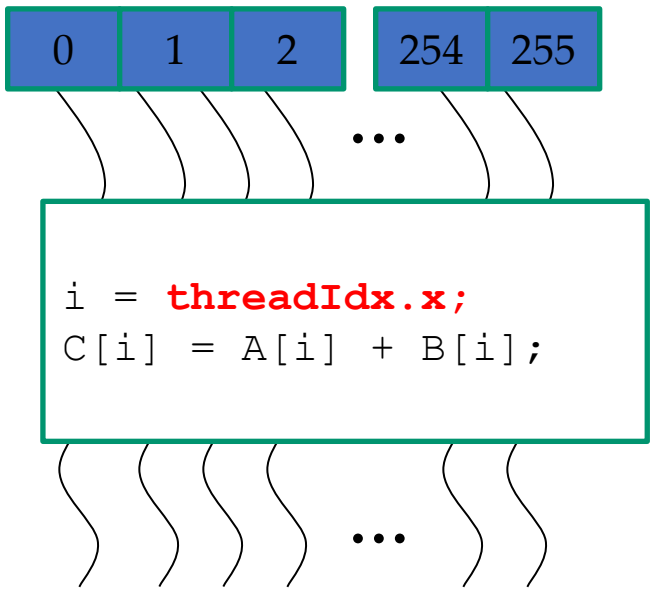
CUDA/OpenCL Execution Model

- **Integrated Host + Device Application (C Program)**
- **Host Code (C):**
  - Handles serial or modestly parallel tasks
- **Device Kernel (C, SPMD Model):**
  - Executes highly parallel sections of the program



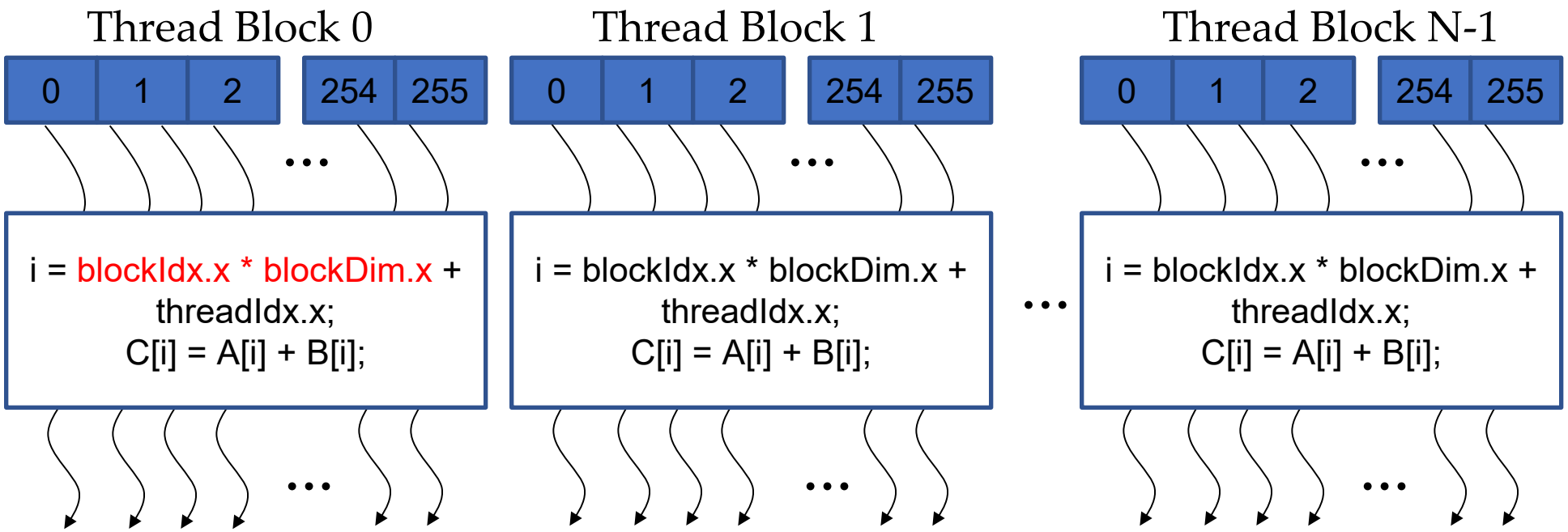
Arrays of Parallel Threads

- A CUDA kernel is executed as a grid (array) of threads
  - All threads in a grid run the same kernel code
  - Single Program Multiple Data (SPMD model)
  - Each thread has a **unique index** that it uses to compute memory addresses and make control decisions



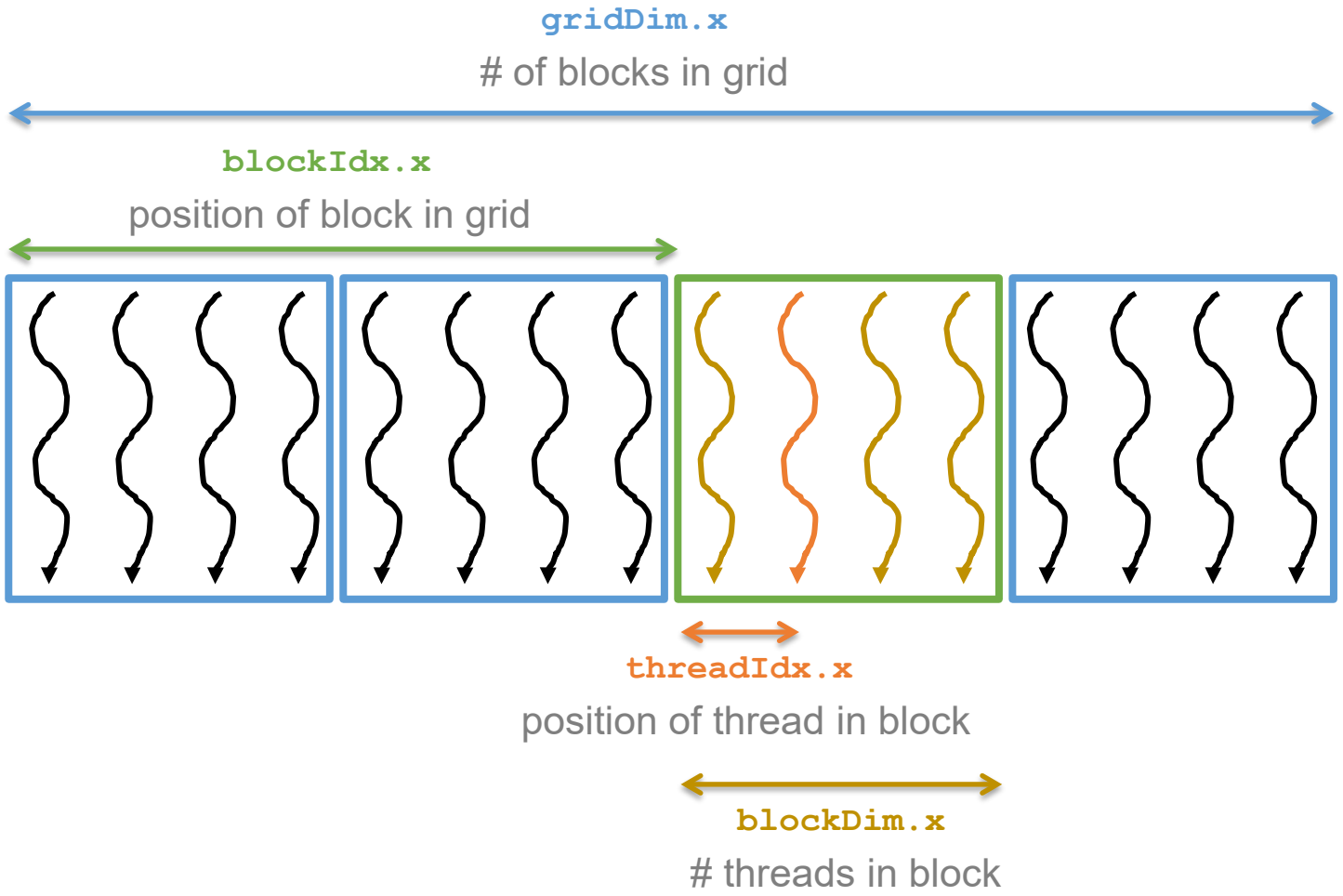
Thread Blocks: Scalable Cooperation

- Threads within a block cooperate via **shared memory, atomic operations** and **barrier synchronization** (to be covered later)
- Threads in different blocks cooperate less.



blockIdx and threadIdx

- Thread block and thread organization
  - simplifies memory addressing
  - when processing multidimensional data

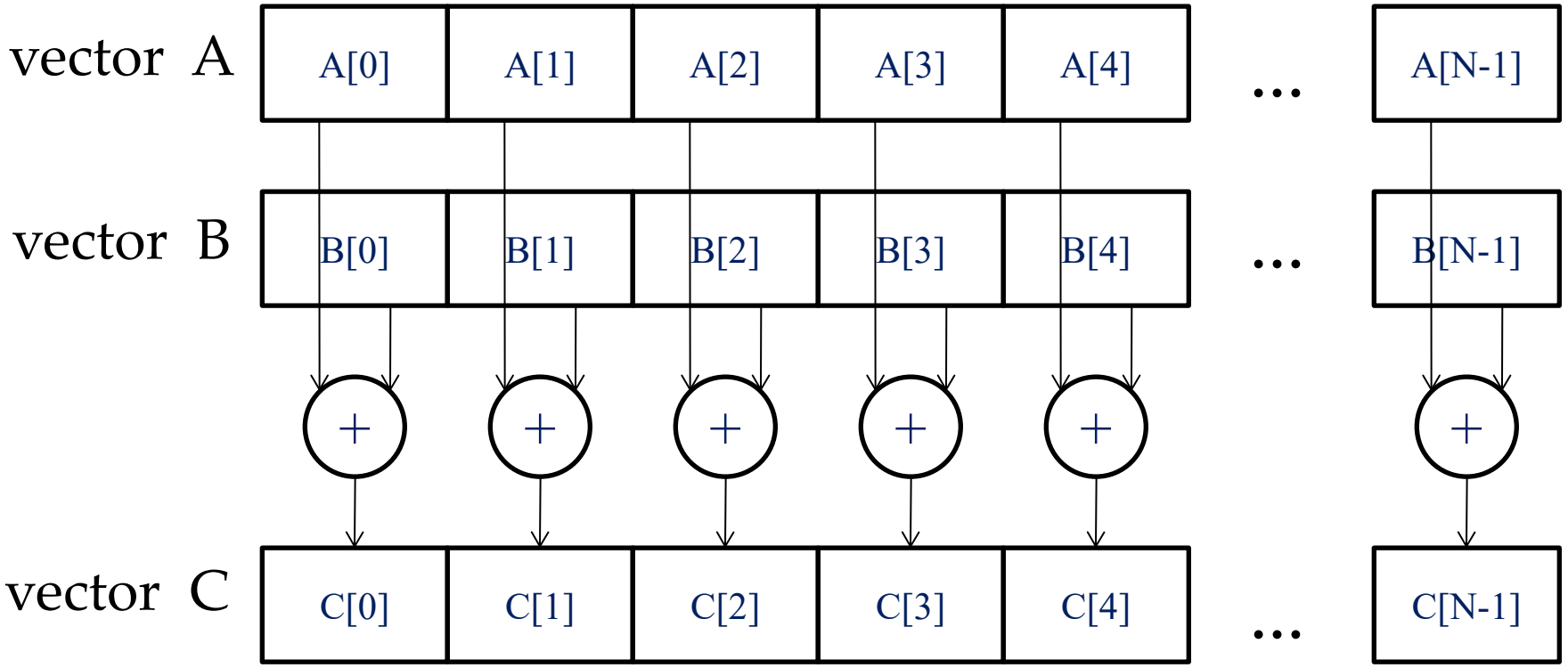


Vector Addition

- The “Hello, World!” for data parallel programming

Input Vector x:	1	2	3	…												
									+							
Input Vector y:	2	3	4	…												
									=							
Output Vector z:	3	5	7	…												

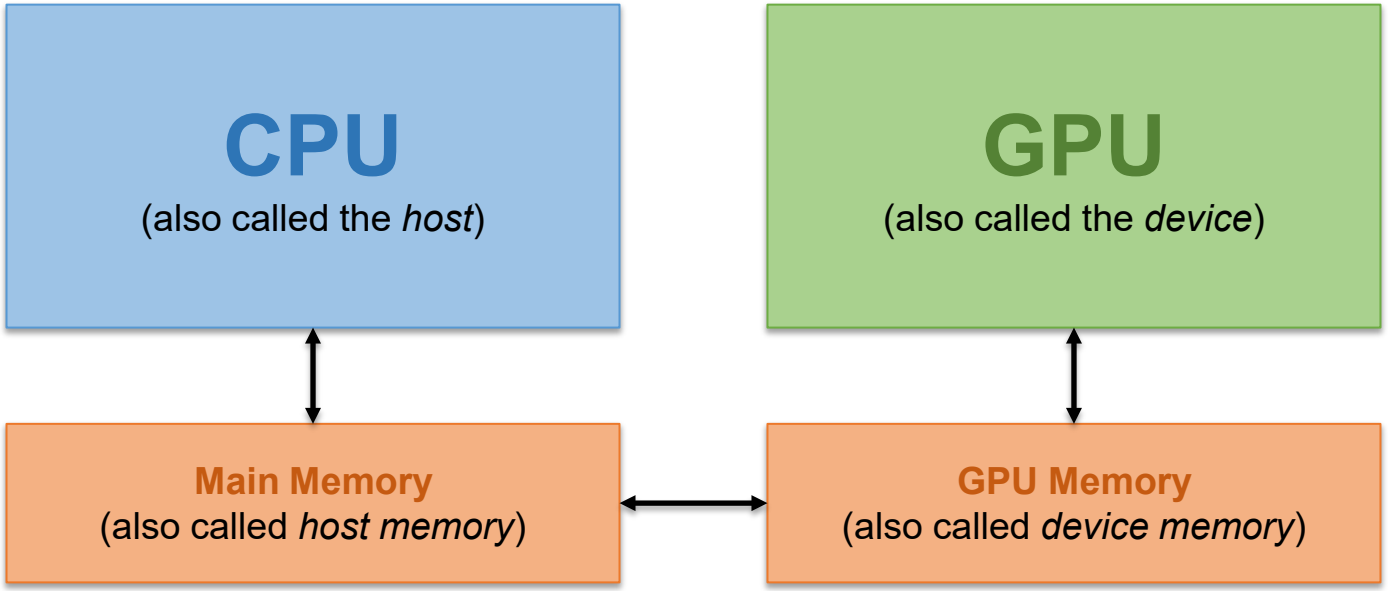
Vector Addition – Conceptual View



Vector Addition – Traditional C Code

```
// Compute vector sum C = A+B
void vecAdd(float* A, float* B, float* C, int n) {
  for (i = 0, i < n, i++) {
    C[i] = A[i] + B[i];
  }
}
int main() {
    // Memory allocation for A_h, B_h, and C_h
    // I/O to read A_h and B_h, N elements
    …
    vecAdd(A_h, B_h, C_h, N);
}
```

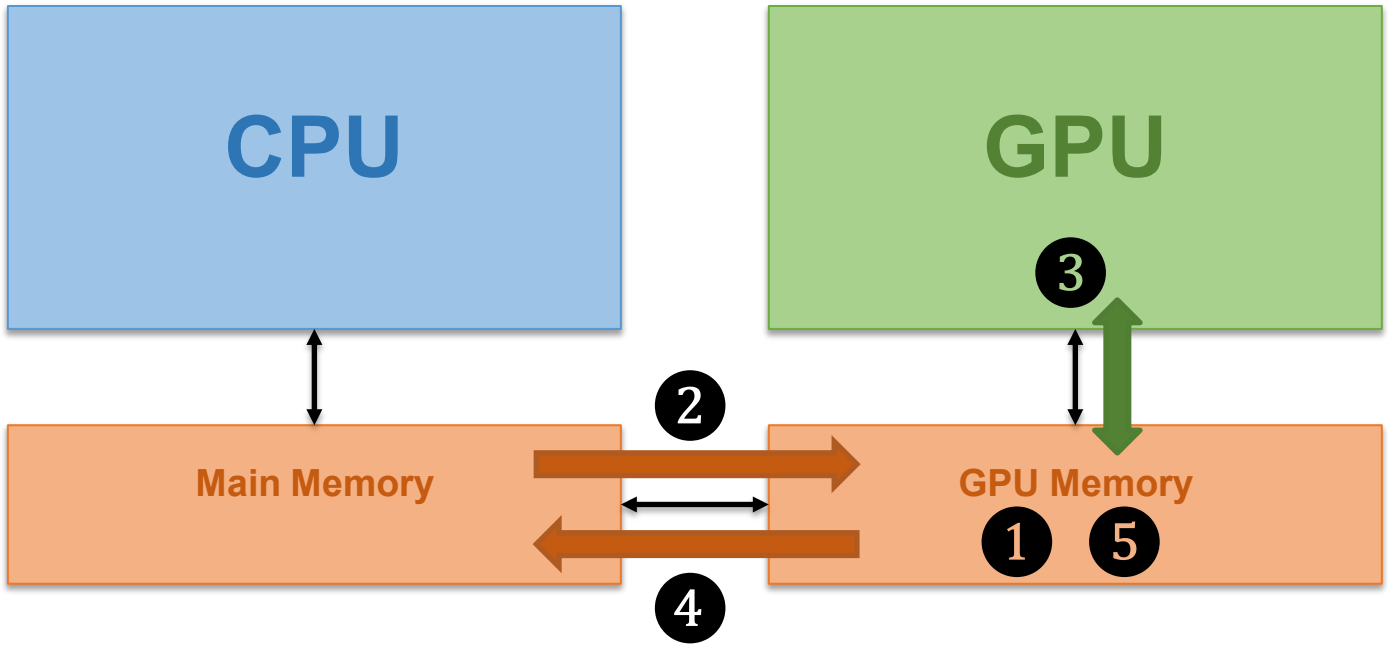
System Organization



The CPU and GPU have separate memories can cannot access each others' memories

Need to transfer data between them

Memory Management



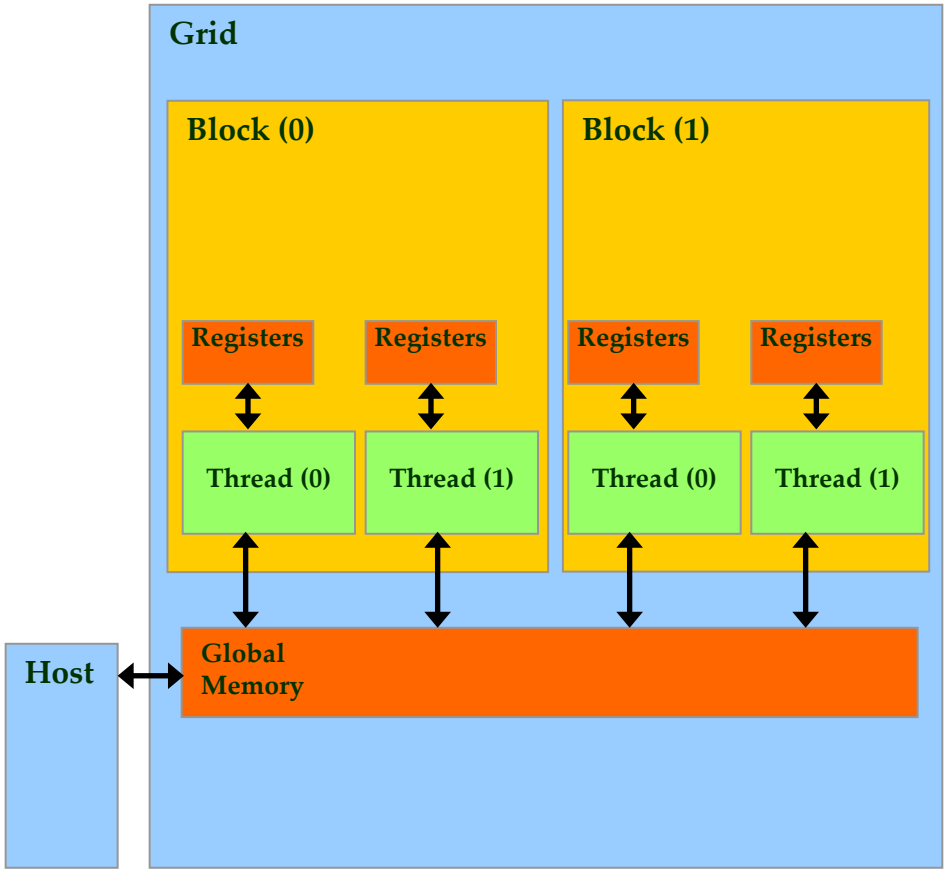
1. Allocate GPU memory
2. Copy data to GPU memory
3. Perform computation on GPU
4. Copy data from GPU memory
5. Deallocate GPU memory

Heterogeneous Computing: vecAdd Host Code

```
#include <cuda.h>

void vecAdd(float* A, float* B, float* C, int n) {
   int size = n* sizeof(float);
   float *A_d, *B_d, *C_d;
    …
1. // Allocate device memory for A, B, and C
   // copy A and B to device memory

2. // Kernel launch code – to have the device
   // to perform the actual vector addition

3. // copy C from the device memory
   // Free device vectors
```

Partial Overview of CUDA Memories

- Device code can:
  - R/W per-thread registers
  - R/W per-grid global memory
- Host code can
  - Transfer data to/from per grid global memory
- We will cover more later.



CUDA Device Memory Management API

- Allocating memory:

```
cudaError_t cudaMalloc(void **devPtr,
size_t size)
```

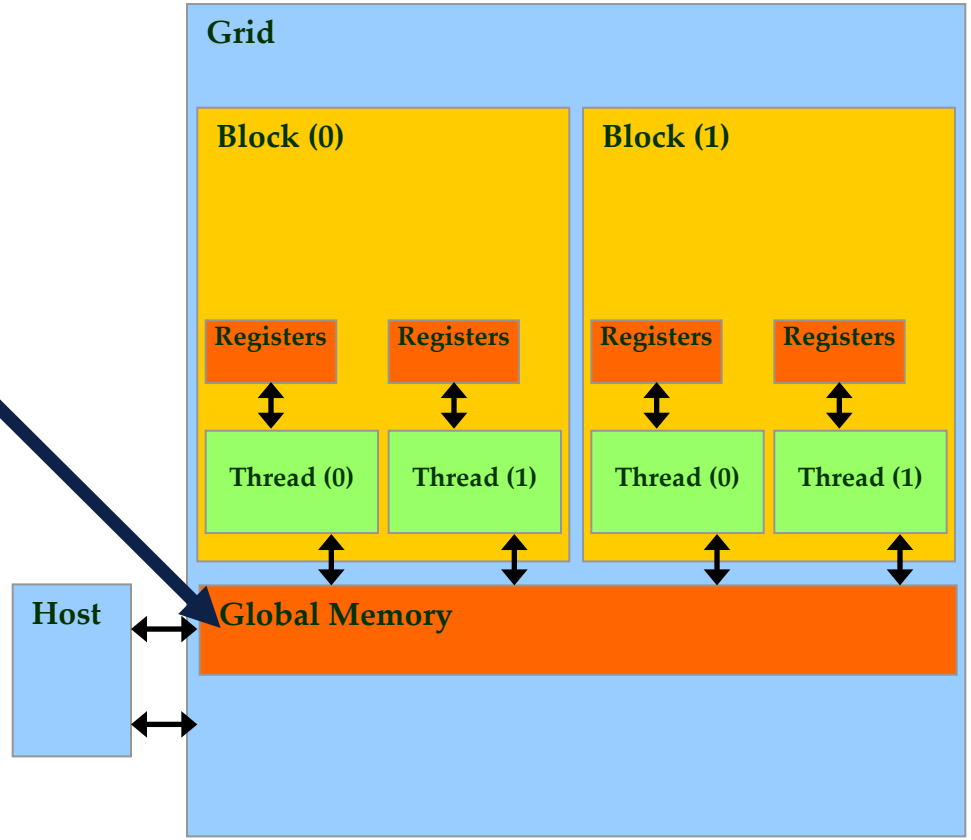
  - `devPtr`: Pointer to pointer to allocated device memory
  - `size`: Requested allocation size in bytes

- Deallocating memory:

```
cudaError_t cudaFree(void *devPtr)
```

  - `devPtr`: Pointer to device memory to free

- Return type: `cudaError_t`
  - Helps with error checking (discussed later)

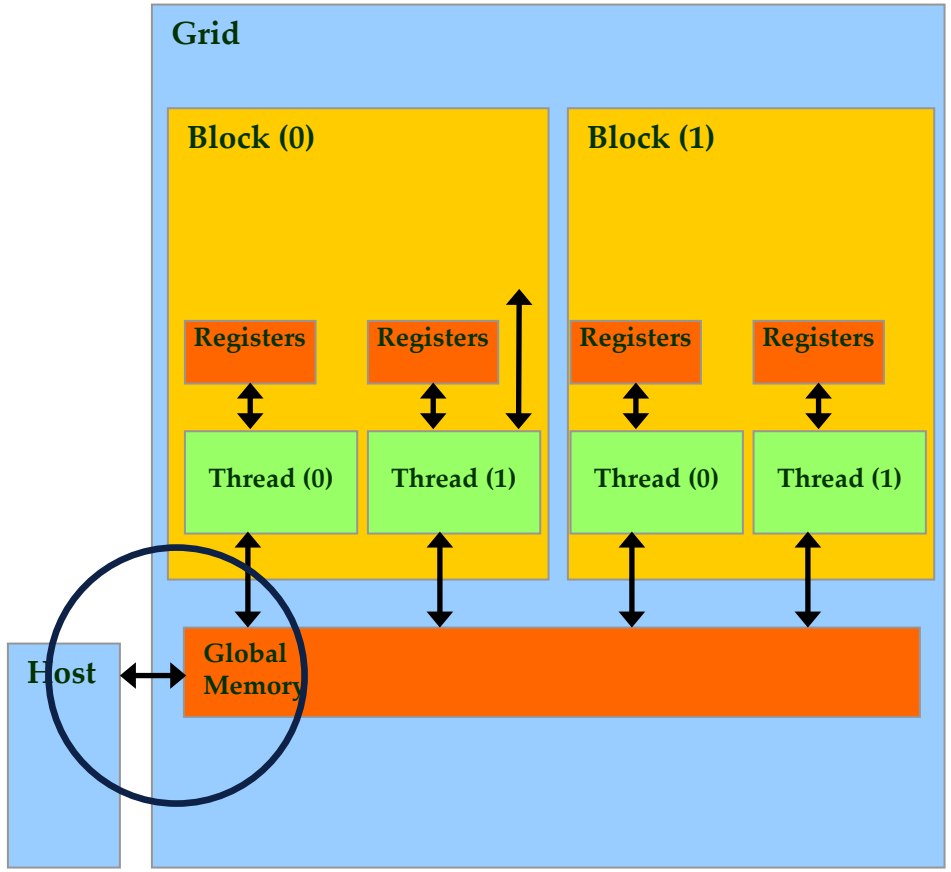


Host-Device Data Transfer API Functions

- Copying memory:

```
cudaError_t cudaMemcpy(void *dst,
const void *src, size_t count,
enum cudaMemcpyKind kind)
```

  - `dst`: Destination memory address
  - `src`: Source memory address
  - `count`: Size in bytes to copy
  - `kind`: Type of transfer
    - `cudaMemcpyHostToHost`
    - `cudaMemcpyHostToDevice`
    - `cudaMemcpyDeviceToHost`
    - `cudaMemcpyDeviceToDevice`

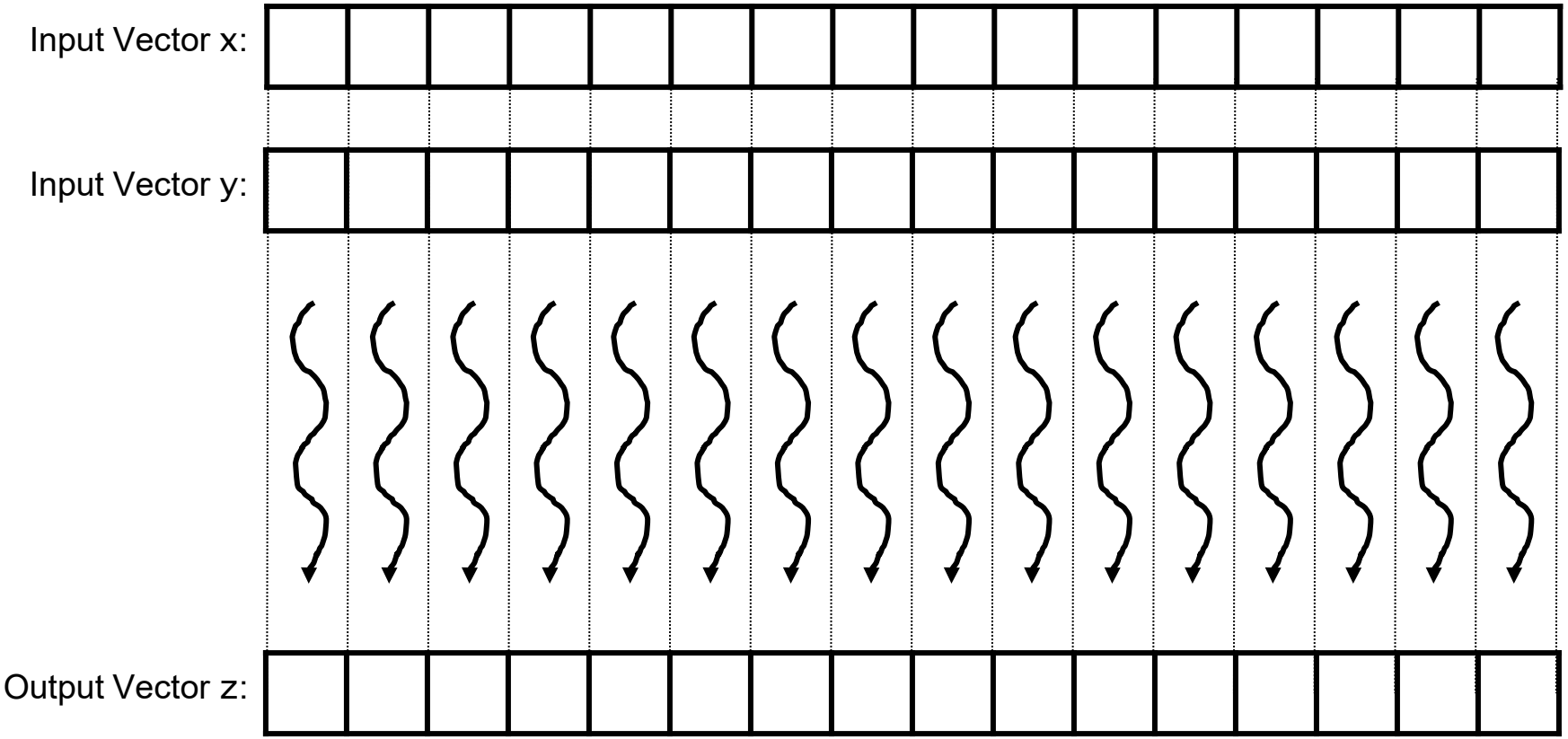


vecAdd Host Code

```
void vecAdd(float* A, float* B, float* C, int n) {
      int size = n * sizeof(float);
      float *A_d, *B_d, *C_d;

1. // Transfer A and B to device memory (error-checking omitted)
    cudaMalloc((void **) &A_d, size);
    cudaMemcpy(A_d, A, size, cudaMemcpyHostToDevice);
    cudaMalloc((void **) &B_d, size);
    cudaMemcpy(B_d, B, size, cudaMemcpyHostToDevice);

   // Allocate device memory for
    cudaMalloc((void **) &C_d, size);

2. // Kernel invocation code – to be shown later
     …
3. // Transfer C from device to host
    cudaMemcpy(C, C_d, size, cudaMemcpyDeviceToHost);
   // Free device memory for A, B, C
    cudaFree(A_d); cudaFree(B_d); cudaFree(C_d);
}
```

Parallel Vector Addition

- Simple strategy: assign one GPU thread per vector element



Launching a Grid

- Threads in the same grid execute the same function known as a **kernel**
- A grid can be launched by calling a kernel and configuring it with appropriate grid and block sizes:

```
const unsigned int numThreadsPerBlock = 512;
const unsigned int numBlocks = n/numThreadsPerBlock;
vecAddKernel <<< numBlocks, numThreadsPerBlock >>> (A_d, B_d, C_d, n);
```

- If `n` is not a multiple of `numThreadsPerBlock`, fewer threads will be launched than desired
  - Solution: use the ceiling to launch extra threads then omit the threads after the boundary:

```
vecAddKernel <<< ceil(n/512.0), 256 >>> (A_d, B_d, C_d, n);
```

More Ways to Compute Grid Dimensions

```
// Example #1
dim3 DimGrid(n/numThreadsPerBlock, 1, 1);
if (0 != (n % numThreadsPerBlock)) { DimGrid.x++; }
dim3 DimBlock(numThreadsPerBlock, 1, 1);

vecAddKernel<<<DimGrid, DimBlock>>>(A_d, B_d, C_d, n);
```

```
// Example #2
const unsigned int numBlocks;
numBlocks = (n + numThreadsPerBlock - 1)/numThreadsPerBlock;

vecAddKernel<<<numBlocks, numThreadsPerBlock>>>(A_d, B_d, C_d, n);
```

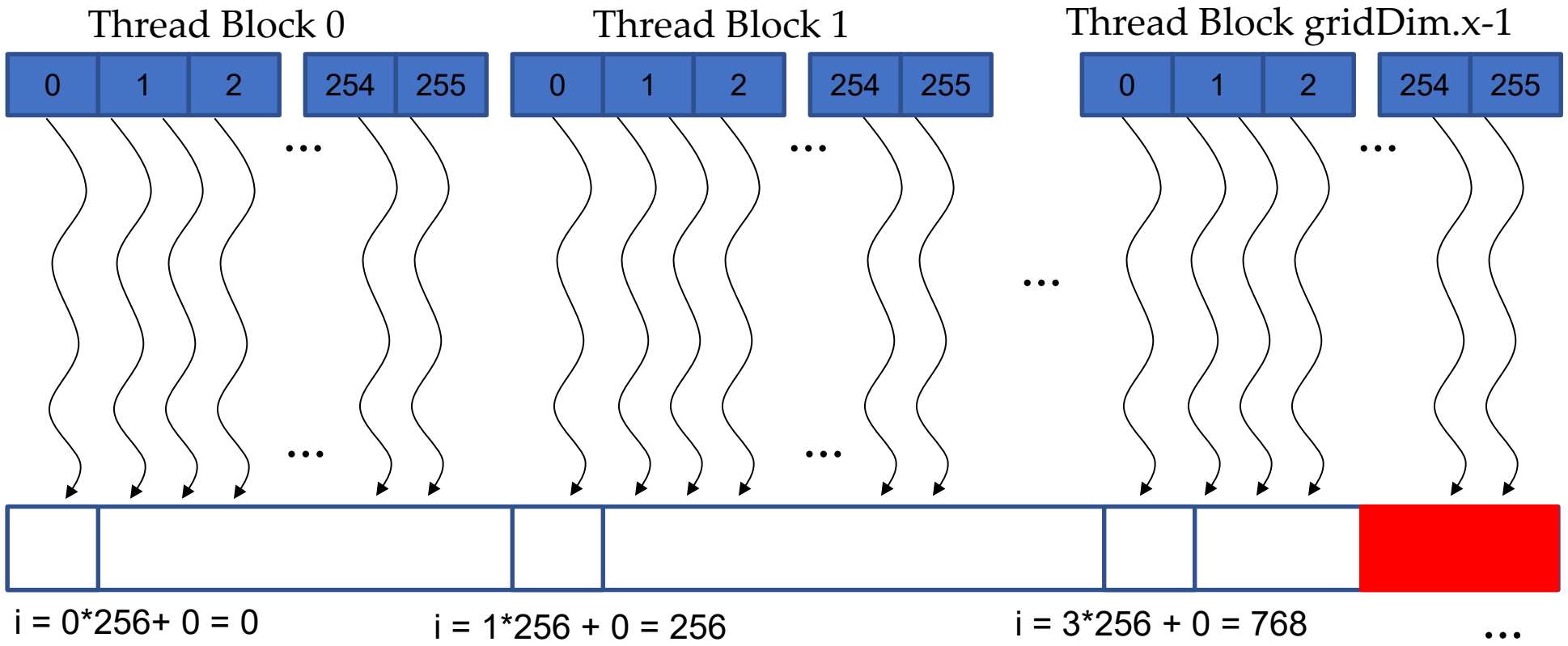
Implementing a Kernel

- A kernel is similar to a C/C++ function
- It is preceded by the keyword `__global__` to indicate that it is a GPU kernel
- It uses special keywords to distinguish different threads from each other

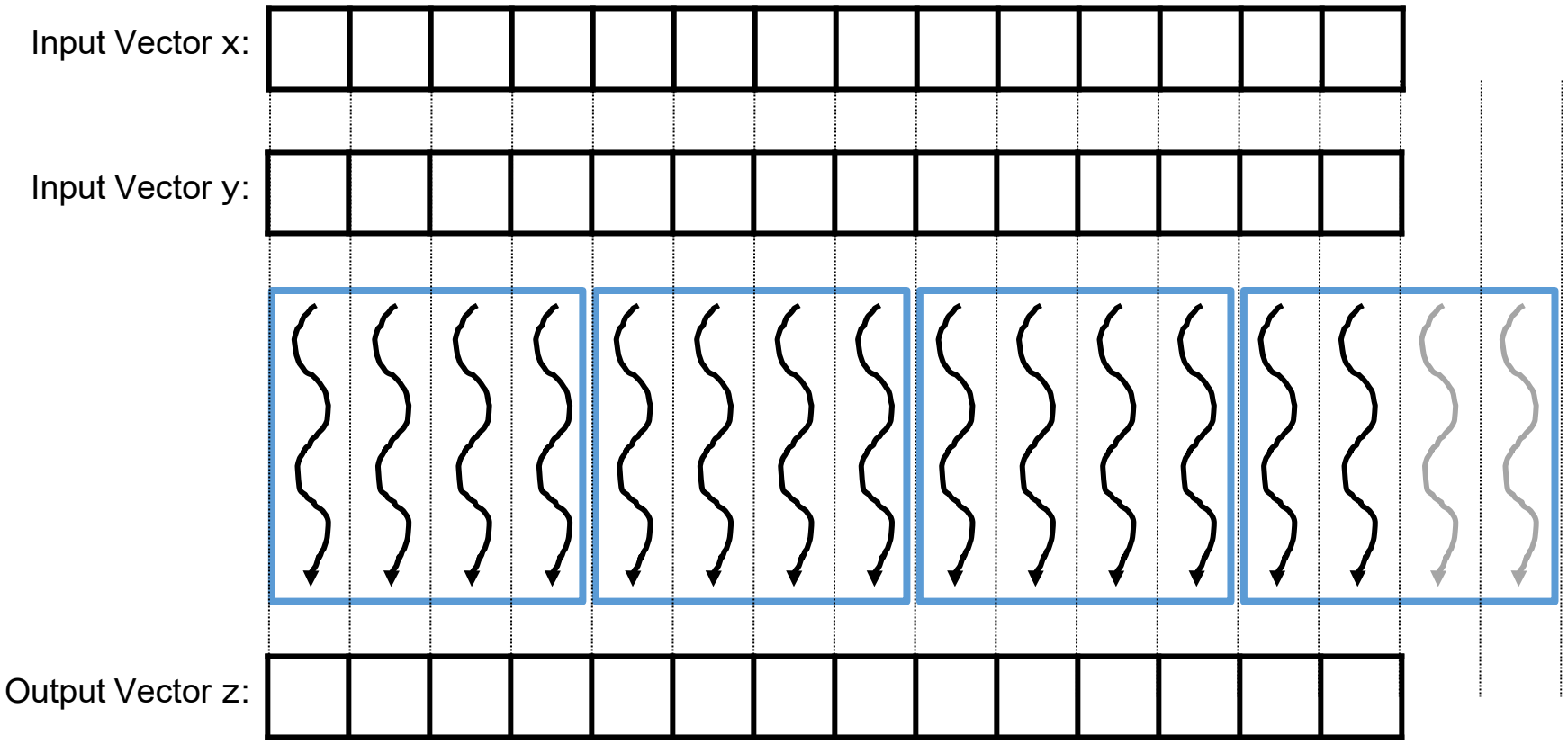
```
// Compute vector sum C = A+B
// Each thread performs one pair-wise addition
__global__
void vecAddKernel(float* A_d, float* B_d, float* C_d, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i<n) C_d[i] = A_d[i] + B_d[i];
}
```

Thread Assignment for vecAdd (n = 1000, block size = 256)

```
vecAddKernel<<<ceil(n/256.0), 256>>>(…)

int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i<n) C[i] = A[i] + B[i];
```



Execution with Boundary Checks



Vector Addition Kernel Review

```
// Compute vector sum C = A+B
// Each thread performs one pair-wise addition
__global__
void vecAddKernel(float* A_d, float* B_d, float* C_d, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i<n) C_d[i] = A_d[i] + B_d[i];
}

int vecAdd(float* A, float* B, float* C, int n)
{
 // A_d, B_d, C_d allocations and copies omitted
 // Run ceil(n/256) blocks of 256 threads each
  dim3 DimGrid(ceil(n/256.0), 1, 1);
  dim3 DimBlock(256, 1, 1);

  vecAddKernel<<<DimGrid,DimBlock>>>(A_d, B_d, C_d, n);
}
```

C D E

A B

A Number of blocks per dimension

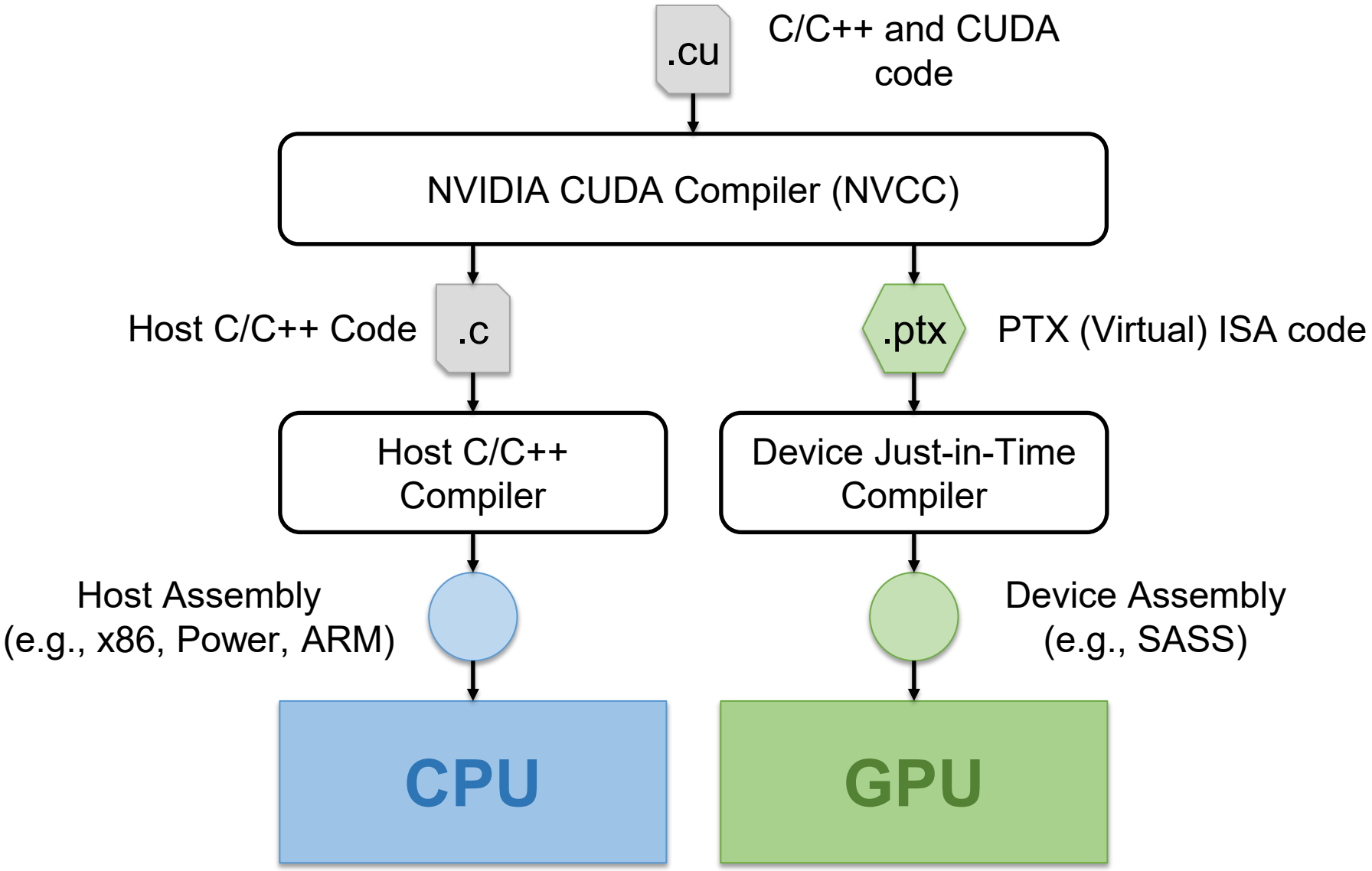
B Number of threads per dimension in a block

C Unique block # in x dimension

D Number of threads per block in x dimension

E Unique thread # in x dimension in the block

Compiling A CUDA Program



Function Declarations in CUDA

- Keywords that differentiate where a function is intended to run:

Keyword	Callable From	Executed On
__host__ (default)	Host	Host
__global__	Host (or Device)	Device
__device__	Device	Device

- `__global__` defines a kernel function
  - Each "__" consists of two underscore characters
  - A kernel function must return `void`
- `__device__` and `__host__` can be used together

More on Function Declarations

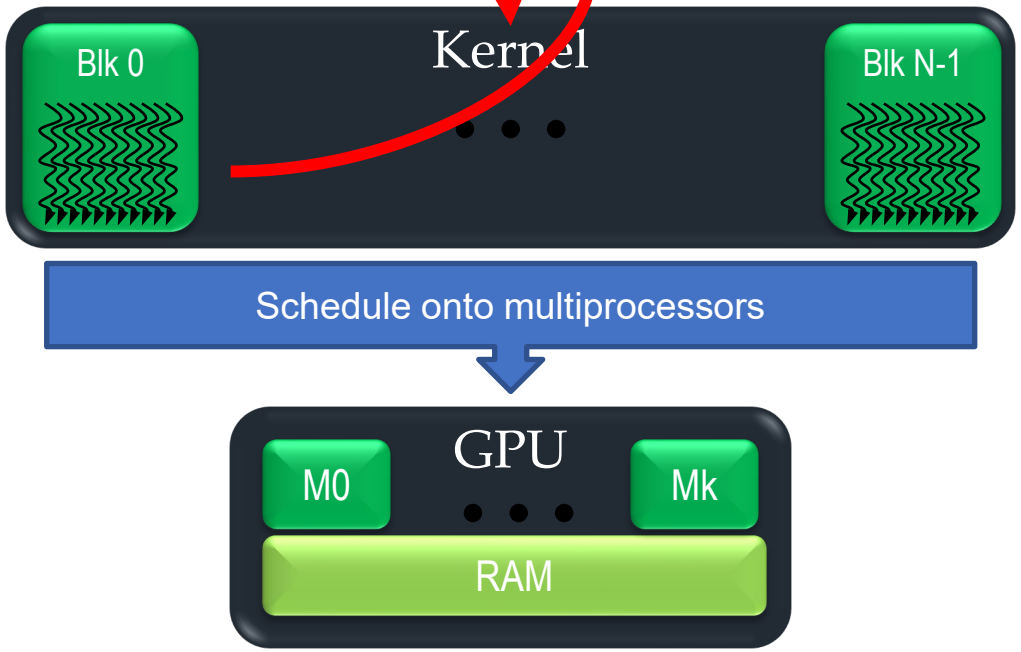
- The keyword `__host__` is useful when needing to mark a function as executable on both the host and the device

```
__host__ __device__ float f(float a, float b) {
    return a + b;
}
void vecadd(float* x, float* y, float* z, int N) {
    for(unsigned int i = 0; i < N; ++i) {
        z[i] = f(x[i], y[i]);
    }
}
__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {
    int i = blockDim.x*blockIdx.x + threadIdx.x;
    if (i < N) {
        z[i] = f(x[i], y[i]);
    }
}
```

Kernel Execution in a Nutshell

```
__host__
void vecAdd()
{
  dim3 DimGrid(ceil(n/256.0),1,1);
  dim3 DimBlock(256,1,1);

vecAddKernel<<<DimGrid,DimBlock>>>
(A_d,B_d,C_d,n);
}
```

```
__global__
void vecAddKernel(float *A_d,
     float *B_d, float *C_d, int n)
{
   int i = blockIdx.x * blockDim.x
     + threadIdx.x;

   if( i<n ) C_d[i] = A_d[i]+B_d[i];
}
```



Asynchronous Kernel Calls

- By default, kernel calls are asynchronous
  - Useful for overlapping GPU computations with CPU computations

- Use the following API function to wait for the kernel to finish

  `cudaError_t cudaDeviceSynchronize()`
  - Blocks until the device has completed all preceding requested tasks

- Better ways of timing kernel execution:
  - Use CUDA events and stream synchronization
  - Use a profiler
  - We will cover these concepts later

Error Checking

- All CUDA API calls return an error code `cudaError_t` that can be used to check if any errors occurred
  - Example:

```
cudaError_t err = ...;
if (err != cudaSuccess) {
    printf("Error: %s\n", cudaGetErrorString(err));
    exit(0);
}
```

- For kernel calls, one can check the error returned by `cudaDeviceSynchronize()` or call the following API function: `cudaError_t cudaGetLastError()`

Any More Questions?

Things to Read:

Textbook chapter 2

CUDA PG: Programming Model

Things to Do:

Get Delta account

Setup GitHub

Submit Lab 0

Problem Solving

- Consider the following code:

```
kernel<<VECTOR_N, ELEMENT_N>>>(d_C, d_A, d_B, ELEMENT_N);
```

- Q: How many CUDA threads are in each block as the result of the following kernel call?
- A: **`ELEMENT_N`**

- Q: How many CUDA threads will be created as the result of the following kernel call?
- A: **`VECTOR_N * ELEMENT_N`**

Problem Solving

- Q: For a vector addition, assume that the vector length is 16000, each thread calculates 8 output elements, and the thread block size is 256 threads. The programmer configures the kernel launch to have a minimal number of thread blocks to cover all output elements. How many **threads** will be in the **grid**?
- A:
  - How many threads do we need? 16000/8 = 2000
  - How many blocks of threads do we need to run 2000 threads? ceil(2000/256) = 8
  - Thus, how many threads will be running? 8 * 256 = 2048

Problem Solving

- Q: A CUDA kernel is launched with 512 thread blocks each of which has 256 threads. If a variable is declared as a local variable in the kernel, how many versions of the variable will be created through the lifetime of the execution of the kernel?
- A:
  - How many threads will be created? 512 * 256 = 131072
  - So, there will be as many copies of the local variable, one in each thread.